



Module 17-1

Generating Test Stimulus

cādence®

This module examines some common coding styles and methods applicable to testbench generation.

Module Objective

In this module, you:

- Generate a test stimulus

Topics

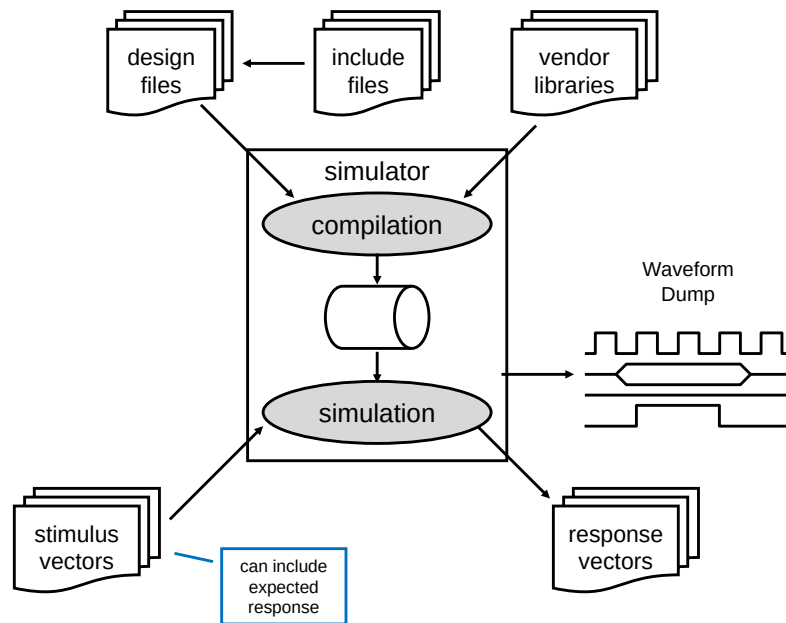
- Simulation process review
- Organizing the testbench
- Using hierarchical names
- Generating clocks
- Generating stimulus
 - “In-line” stimulus
 - Stimulus using loops
 - Stimulus using tasks
 - Testing “corner” conditions



Your objective is to generate test stimulus.

To do that, you need to know how to create a testbench and some common ways to generate stimulus.

Simulation Inputs and Outputs Review



3

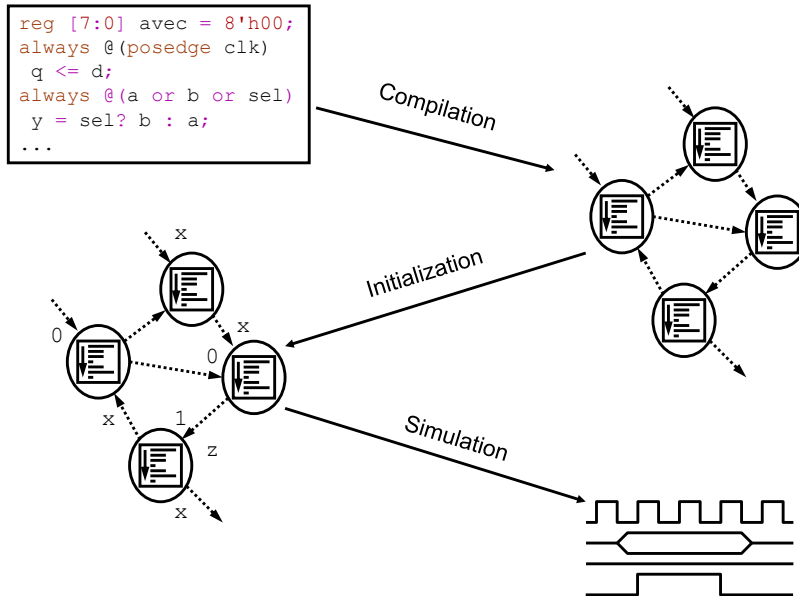
© Cadence Design Systems, Inc. All rights reserved.



Input to the compiler is source text, which may reference third-party IP, and potentially other files to define the design configuration. Output from the compiler is a database representation of the design hierarchy.

Input to the simulator is of course the database representation of the design hierarchy, and can include stimulus vectors, may include expected response vectors, or perhaps some sort of pseudocode to drive an on-board stimulus generator. Output from the simulator is whatever diagnostic messages the user coded and value change data from whatever nets and variables the user probed.

Simulation Process Review



4

© Cadence Design Systems, Inc. All rights reserved.

cadence

Simulation of the model takes the following steps:

- The 1st step is to compile the model:
 - The compiler parses the source text and constructs an internal database object for each design unit. A separate part of the compilation process, called elaboration, then links together the various parts of the design and builds a data structure representing the design hierarchy. For precompiled implementations, yet another process, called code generation, generates platform executable code for the design unit behaviors.
- The 2nd step is to initialize the elaborated data structure:
 - The initialization phase initializes most nets to the high impedance state and initializes most variables to the unknown state. It initializes trireg nets to the unknown state and real variables to 0.
- The 3rd step is the actual simulation, starting at time 0:
 - The simulation first propagates the design initialization events through the design hierarchy. The simulation then executes the statements in each initial and always construct up to the point at which a timing control blocks execution or the initial construct completes. Assignments in these statements can schedule events for the current and future times. As time advances, the simulator executes scheduled events, this execution often scheduling additional events. This process continues throughout the simulation.

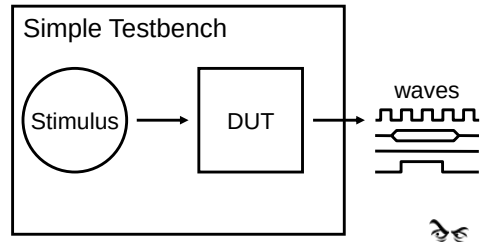
The simulator initializes most variables to the unknown state. Variables remain at the unknown state until an assignment statement assigns some other state. A variable remaining in the unknown state throughout the simulation is an indication of a failure to assign a known state to the variable, most likely a failure to reset the design.

Most nets initialize to the high impedance state. A net remaining at the high impedance state throughout the simulation is an indication that nothing drives the net.

Organizing the Testbench

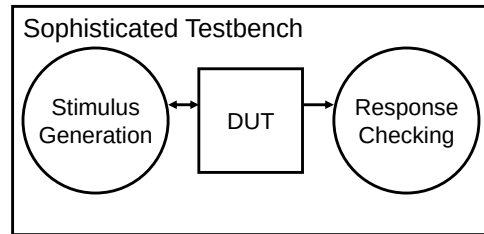
Simple testbench

- Just sends data to design
- Few procedures
- No interaction



Sophisticated testbench

- Models system-level design environment
- Two-way communication
 - Stimulus generation
 - Response checking



We are most interested in sophisticated testbenches!

A simple testbench applies vectors to the design under test (DUT) and the user manually verifies the results.

Non-trivial projects use some form of “intelligent” or “smart” testbench that reacts to the DUT, e.g., for a bus protocol that requires some kind of handshake mechanism.

Such sophisticated testbenches are self-checking, i.e., they automatically verify the results.

Sophisticated testbenches typically instantiate modules for stimulus generation and response checking under a top-level “wrapper”, to more easily swap different tests and design configurations “in” and “out”.

Using Hierarchical Names

Every identifier has a unique hierarchical path name.

- Starts at a top module instance and downward traverses module instances, named blocks and tasks and functions.
- Each scope separated by “dot” (“.”).
- Hierarchical names can be used to access identifiers in other modules.
 - Also called Out Of Module References (OOMRs)
- There are restrictions:
 - Cannot access items of automatic tasks or functions
 - Cannot access items of unnamed blocks
- Hierarchical pathnames can be used anywhere but are primarily used in testbenches.

```
module fsm (
    input clk, rst, do,
    output done);
    reg [2:0] state;
    // assume fsm behaviors
endmodule

module fsm_tb;
    reg clk, rst, do; wire done;
    fsm u1 (clk, rst, do, done);
    initial begin
        $monitor($stime,,u1.state);
        // assume test behaviors
    end
endmodule
```

Path name can be:

- Downward relative to current scope.
- Downward starting at higher module or instance.

Every identifier has a unique hierarchical path name. The path name starts at a top module instance and traverses module instances, named blocks and tasks and functions, downward. Each such scope in the path name is separated from the next with the dot (.) character. You can make hierarchical references from anywhere in the design to almost anywhere else in the design. The exceptions are that you cannot make hierarchical references to items of automatic tasks and automatic functions, and you can make hierarchical references to items of an unnamed generate block only from within that generate block.

You can access items within the hierarchy rooted at the current scope by using relative path names. You access other items using absolute path names. The example testbench monitors the internal state of the FSM. This is a common use of hierarchical references – to gain testbench access to signals that are not available through module ports.

Hierarchical pathnames are primarily for testbench use, but can also be used in cross-hierarchy references, to access identifiers in one sub-module from another. As hierarchical pathnames cannot contain explicit “up-scope” references, cross-hierarchy references rely on the following resolution process:

- Hierarchical paths do NOT start at the top module instance.
- The paths are always relative to where they are used.
- If there is no match for the path downwards from the current scope, then the simulator automatically jumps up one scope and looks for a match from there.
- If there is still no match, then the simulator keeps jumping up until the top level module.
- If there is still no match from there, then the path is invalid and an elaboration error is produced.

If you use hierarchical names in testbenches only (as recommended) then pathname resolution is not a problem. The resolution is only an issue when you embed pathnames in hierarchical blocks, which we generally discourage (since it breaks reuse).

Generating Clocks

Behavioral Clocks

```
reg clk = 1;
always // simple
#(PERIOD/2) clk = ~clk;
```

```
reg clk = 1;
initial // delayed
#(DELAY) forever
#(PERIOD/2) clk = ~clk;
```

```
reg clk = 1;
initial // irregular
#(DELAY) forever
begin
  #(( DUTY)*PERIOD) clk = 0;
  #((1-DUTY)*PERIOD) clk = 1;
end
```



```
always @(clk)
#5 clk = !clk; //doesn't oscillate

always
clk <= #5 !clk; //zero delay loop
```



The master clock generator is seldom part of the design and many people place it in the top-level wrapper, because it is also not really part of the stimulus generation or response checking operations. For a typical large and complex design, efficiency of the clock generator itself is not usually a concern.

In the “Beware” example section, the coding style to generate clock does not oscillate the clock and can lock the simulator.

Generating Stimulus

This section examines stimulus generation:

- The incremental approach
- Generating in-line stimulus
- Generating stimulus using loops
- Generating stimulus using tasks
- Testing “corner” conditions



This section examines stimulus generation. This introduction to test generation gives you a pretty good idea of what you can do with Verilog to test your design. The SystemVerilog language, which is a superset of Verilog, has much more powerful features directly targeting test generation. Industry advocates provide the Universal Verification Methodology (UVM), based on SystemVerilog, and plug-and-play verification IP components with which you can quickly construct a sophisticated verification environment.

The Incremental Approach

One of your most valuable test strategies is to the incremental approach:

- Build incrementally on what you know works:
 - First test for signs of life
 - Then, test interface and basic functions
 - Finally, test internal complex functionality
- For example, test of a storage device might:
 - Test reset (if that feature exists)
 - Test data lines individually or in combinations
 - Test address lines individually or in combination
 - Test functional “corners” i.e., back-to-back and overlapping operations
 - Test timing “corners” i.e., frequency, setup/hold, recovery/removal



A test sequence may in general incrementally contain elements from the following:

- Check-in tests, to verify the model before checking it into source control.
- Connectivity tests of the model, ASIC or other subunit instantiation.
- Functional tests of major widely-used functional features.
- Deterministic worst-case tests on data and control.
- Random worst-case tests on data and control.
- Tests of asynchronous interactions, including clock/data margining.
- Tests of inter-ASIC datapath and data processing algorithms.
- Hardware subsystems test using basic system-level operations and protocols.
- System boot.
- System multitasking and exception handling.
- Randomly sequenced full-system tests with heavy workload.

Generating In-Line Stimulus

The simplest approach is to use in-line stimulus generation:

- You only need to specify the variable value transitions.
- You can easily specify complex timing relationships.
- This testbench can become very large for complex tests.

```
module inline_tb;
  reg rd=0, wr=0;
  reg [4:0] addr;
  reg [7:0] dreg;
  wire [7:0] data=dreg;
  mem u1 (addr,data,rd,wr);
  initial
  begin
    #10 wr=1; addr=8'h00; dreg=8'h00;
    #10 wr=0;
    #10 rd=1; addr=8'h00; dreg=8'hZZ;
    #10 rd=0;
    ...
  end
endmodule
```



The simplest way to generate a stimulus is with an in-line sequence of assignments.

This works reasonably well for an individual test with an irregularly-timed control protocol, as you specify exactly the transitions you need and exactly the time that you need each one.

This testbench can become very large for complex tests. When you see this type of stimulus, it is almost always machine-generated stimulus derived from dumped test vectors.

This example does one memory write operation followed by one memory read operation.

Generating Stimulus Using Loops

For repeated stimulus using a regularly-timed control protocol, you can “roll” your inline stimulus into loops. Advantages include:

- Compact easier to understand and debug stimulus generation.

```
module loop_tb;
  reg rd=0, wr=0;
  reg [4:0] addr;
  reg [7:0] dreg;
  wire [7:0] data=dreg;
  mem ul (addr,data,rd,wr);
  initial
  begin: TEST
    integer i;
    for (i=0;i<=31;i=i+1)
      begin
        #10 wr=1; addr=i; dreg=i;
        #10 wr=0;
      end
    for (i=0;i<=31;i=i+1)
      begin
        #10 rd=1; addr=i; dreg=8'hZZ;
        #10 rd=0;
      end
    end
endmodule
```

For repeated stimulus using a regularly-timed control protocol, you can “roll” your inline stimulus into loops. This produces stimulus generation code that is more compact and easier to understand and debug.

This example uses loops to first fill a memory with data and then to examine the data in memory.

Even very sophisticated testbenches use loops to generate stimulus, but they also take advantage of other techniques that we will examine next.

Generating Stimulus Using Tasks

Use a task if:

- You need the same sequence of control signals more than once.
- Or even if you just want to encapsulate them.

```
task write ( input [4:0] a,
             input [7:0] d );
begin
  #10 wr=1; addr=a; dreg=d;
  #10 wr=0;
end
endtask

task read ( input [4:0] a,
            output [7:0] d );
begin
  #10 rd=1; addr=a; dreg=8'hZZ;
  #10 rd=0; d=data;
end
endtask

initial begin: TEST
  integer i;
  for (i=0;i<=31;i=i+1)
    // call tasks
  end
```

Placing operations in tasks enhances readability, and if the operations are required in more than one place, it is essential for compact and concise code.

This illustration encapsulates the write and read operations in tasks, which procedural statements anywhere in the design can call upon.

Testing Boundary Conditions

Test boundary conditions with worst-case tests that do the following:

- Test initial and terminal conditions.
- Focus on where the problems are.

Two Illustrations

```
reg [1:`WIDTH] memory [1:`SIZE];  
...  
for (i=1; i<=NUMBER; i=i+1)  
  memory[i] = memory[i+1];
```

What is the architect's intent
if NUMBER==0 ?

What is the architect's intent
if NUMBER==`SIZE ?

```
always @(MEM_ACCESS)  
  case ({READ, WRITE})  
    1: memory[location] = ibus;  
    2: obus = memory[location];  
    3: ???  
  endcase
```

What happens if READ and
WRITE are both true?



The illustrations indicate what is meant by a worst-case condition. Every design has its own set of worst-case conditions: A queue becomes full or empty; an arbiter receives simultaneous requests. The set of worst-case conditions can be very large, and at the beginning of the project, not well understood.

Example Worst-Case Test – FIFO Model

```

module fifo ( output [1:8] data_o, output full, empty,
              input [1:8] data_i, input load, unload, rst, clk );
  reg [1:8] busy, fifo[1:8];
  integer i;
  assign data_o = fifo[1], full = busy[8], empty = ~busy[1];
  always @(posedge clk)
    if (rst)
      busy = 0;
    else
      case ({load, unload})
        2'b01: for (i=1; i<=8; i=i+1) // - UNLOAD ONLY -
          begin
            fifo[i] <= (i==8)? 8'bx : fifo[i+1];
            busy[i] <= (i==8)? 1'b0 : busy[i+1];
          end
        2'b10: for (i=8; i>=1; i=i-1) // - LOAD ONLY -
          begin
            fifo[i] <= busy[i]? fifo[i] : data_i;
            busy[i] <= (i==1)? 1'b1 : busy[i-1];
          end
        2'b11: for (i=1; i<=8; i=i+1) // - LOAD AND UNLOAD -
          fifo[i] <= (i==8)? data_i : busy[i+1]? fifo[i+1] : data_i ;
      endcase
  endmodule

```



1	2	3	4	5	6	7	8	b
2								u
3								s
4								y
5								b
6								i
7								t
8								s

The model treats the first and last locations differently than all other locations. Its treatment of the first and last locations is thus more susceptible to error, and the testbench must focus on testing model accuracy around those locations.

The model also becomes more complex in the area where it handles simultaneous read and write operations. This is an area that the model is more likely to inaccurately define, and the testbench is more likely to miss.

Example Worst-Case Test – FIFO Test Tasks

```

module test ( output reg [1:8] data_i, output reg load, unload, rst,
              input wire [1:8] data_o, input wire full, empty, clk );

// - TASK TO CHECK FIFO DATA -
task checkit (input full_e, empty_e, input [1:8] data_e);
  if ( {full, empty} != {full_e, empty_e}
      || !empty_e && data_o != data_e ) begin
    $display("ERROR- FIFO TEST FAILED");
    $finish;
  end
endtask

// - TASK TO LOAD FIFO -
task loadit (input [1:8] data, input full_e, empty_e, input [1:8] data_e);
  begin
    @(negedge clk) load = 1; data_i = data;
    @(negedge clk) load = 0; data_i = 8'hz;
    checkit (full_e, empty_e, data_e);
  end
endtask

// - TASK TO UNLOAD FIFO -
task unloadit (input full_e, empty_e, input [1:8] data_e);
  begin
    @(negedge clk) unload = 1;
    @(negedge clk) unload = 0;
    checkit (full_e, empty_e, data_e);
  end
endtask

```

The testbench defines three tasks to off-load repetitive work from the main test block:

- The loadit task presents data to the FIFO and pulses the load signal.
- The unloadit task pulses the unload signal.
- The checkit task compares flags and data with expected values.

Example Worst-Case Test – FIFO Test Sequence

```

task resetit;
begin
  load = 0; unload = 0;
  @(negedge clk) rst = 1;
  @(negedge clk) rst = 0;
  checkit (0, 1, 8'hX);
end
endtask

initial begin: TST
  integer i;
  resetit;
  loadit (1, 0, 0, 1); // Empty, not full
  loadit ( 8, 1, 0, 1); // Not empty, 1st data
  for (i=2; i<=7; i=i+1) loadit (i, 0, 0, 1); // Not yet full, still 1st data
  loadit ( 8, 1, 0, 1); // Now full, still 1st data
  loadit (-1, 1, 0, 1); // Overflow ignored
  unloadit (0, 0, 2); // No longer full, 2nd data
  for (i=3; i<=8; i=i+1) unloadit (0, 0, i); // Not yet empty, most data
  unloadit (0, 1, 8'hX); // Now empty
  unloadit (0, 1, 8'hX); // Underflow ignored
  $display ("FIFO TEST PASSED");
  $finish;
end

endmodule

```

This sequence tests several FIFO boundary conditions. The test sequence verifies that the FIFO operates correctly, and also verifies that the FIFO does not operate incorrectly. The test sequence verifies:

- That the model sets the full flag only when the FIFO is full, and clears it only when the FIFO becomes not full.
- That the model sets the empty flag only when the FIFO is empty, and clears it only when the FIFO becomes not empty.
- That the model will not load data into a full FIFO, i.e., it does not change flags or data.
- That the model will not unload data from an empty FIFO, i.e., it does not change flags or data.

Module Summary

You should now be able to generate test stimulus.

This module discussed:

- Using hierarchical names
- Generating clocks
- Generating “in-line” stimulus
- Generating stimulus using loops
- Generating stimulus using tasks
- Testing “corner” conditions



This module examined some common coding styles and methods applicable to testbench generation.



Module 17-2

Developing a Testbench

cādence®

This module presents design verification concepts, reinforced with small working examples and illustrations.

Module Objectives

In this module, you:

- Optimize the Testbench Development Effort
- Develop Reusable Testbenches

Topics

- Test Philosophy
- Test Configuration



- Your first objective is to optimize the testbench development effort. The first part of this module supports that objective by exploring the philosophy of system-level test, including the test plan, completion criteria, and design data management.
- Your second objective is to develop reusable testbenches. The second part of this module supports that objective by presenting self-checking tests and various ways to configure the test environment.

Test Philosophy

This section discusses:

- The design verification challenge
- The design verification plan
- The design verification goal
- System-level test
- Completion criteria:
 - Coverage metrics
- Managing the design space
- Testing design regression
- Self-checking test



This section addresses several aspects of test philosophy.

The Design Verification Challenge

The design verification challenge is a resource issue.

Problem

- Technological advances drive system complexity.
- Technology alone cannot overcome the complexity problem.
- Verification is majority of total development effort... *and growing!*
- Integrity means design/implementation team is **NOT** verification team!
 - Although... an early implementer can be a verifier later
 - But must adhere to verification team's process and standards!

Solution

- The Design Verification Plan:
 - Know what you need and when you need it

We need a plan...

21 © Cadence Design Systems, Inc. All rights reserved.



Design verification ensures that the design continues to comply with all system requirements throughout all levels of abstraction, from the system behavioral level to the block implementation level, and continuing through integration of the physical units.

You will need to integrate, structure, and coordinate your verification methods to minimize the probability that the development team misses errors, especially complex system-level design errors, until much later in the design process. Such large-scale redesign is much more costly than iteration confined to a single design phase.

The industry currently uses event-driven simulation as the most common design verification technology at the behavioral and the functional level. This technology is practical, well understood, and mature, but unable to keep up on its own with Moore's law: that design complexity grows at an exponential rate, approximately doubling every 12 to 18 months.

Faster technologies are available, for example, hardware acceleration, cycle-based simulation and emulation, but are less applicable at the higher levels of abstraction early in the design process.

The key to the design verification challenge is to not rely on technological advances alone, but to carefully plan and execute the overall design and design verification strategy.

The Design Verification Plan

The design verification plan is an important job.

There are two approaches:

- Just let it happen (it won't) – oops!
- Make it happen!
 - Test in accordance with the functional specification.
 - Paraphrase the spec; spec says “it does this”, test that “it does this”...
 - Remember the spec can be wrong too!
 - Schedule test delivery to coincide with module and model availability.
 - For example: stub (shell) / Bus-Functional Model (BFM) / fully functional model / hardware model.
 - Model availability is often an issue!

As time goes by...



A viable design verification plan must include the following elements:

- 1st – Use of the appropriate technology for each phase of the design process
 - This requires understanding the available technologies, the needs of the project, and the verification strategy. Depending upon the design phase, appropriate technologies can be anything from C-based stochastic analysis to a hardware prototype.
- 2nd – Creation of appropriate, easily-selectable simulation configurations
 - Configurations must be designed to focus verification efforts on key partitions of the design, while filling in the rest of the system context with behavioral models. The number and design of configurations also heavily depends upon the resources available, both of hardware and of personnel.
- 3rd – Development of formal, well-structured testbench templates
 - The use of testbench templates facilitates test generation, maintenance, and modification. Test procedures, test data, and test timing should be separated, and good programming practices, such as parameterization, should be used.
- 4th – Generation of appropriate tests in a timely manner

The Design Verification Goal

The design verification goal is progressive testing.

Design verification is a continuous parallel effort.

- At the behavioral level:
 - Verify implementation of architectural decisions
 - Test sub-block interface and interaction
 - Test timing to within cycle accuracy at sub-block interfaces
- At the functional level:
 - Verify implementation of behavioral models
 - Test complete design functionality
 - Test timing to within cycle accuracy at all levels of abstraction
- At the structural level:
 - Verify synthesis of functional implementation
 - Test subsystem interconnection and initialization
 - Test timing to high accuracy within structural level

We need a system...

23 © Cadence Design Systems, Inc. All rights reserved.

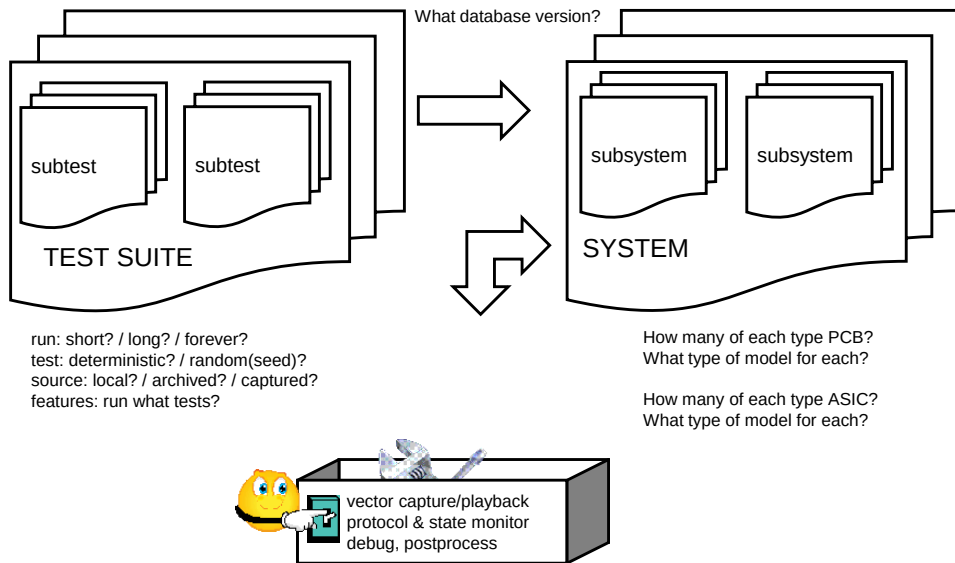


All design verification strategies have these two overall goals:

- The first goal for high-level system design verification is to validate the design intent before implementation is begun. This involves:
 - Verification of system functionality – that algorithm selection is correct.
 - Verification that the system design meets performance and cost requirements.
 - Verification that subsystem partitioning and packaging, into hardware and software, for example, or between physical components, is practical and effective.
- The second goal is to verify the design implementation. This involves:
 - Verification that all components, whether newly designed, reused, or standard, off-the-shelf, work as expected within the context of the new system-level design.
 - Verification of all internal functionality of newly designed subsystems.
 - Verification of hardware and software interaction.

System-Level Test and Configurability

System-level test is about configurability.



24

© Cadence Design Systems, Inc. All rights reserved.

cadence

System-level test is about configurability. A simulation configuration groups specific models of specific subsystems together for a defined verification purpose. The simulation configuration may also constrain the type, detail, and length of tests, or you may wish to postpone such test selection until runtime.

You should include in your testbench:

- Tools for vector capture and playback.
- Monitors of bus protocol, arbiter fairness and state machine transitions.
- Debug tools.
- Postprocessors for the ASIC foundry.

System-Level Test and Team Communication

System-level test is about communication!

There are two approaches:

- System-level test:
 - Developed in parallel with hardware (tested “as a whole” from day 1)
 - Used (with reconfiguration) by everybody
 - Demands up-front planning and continued maintenance
 - Facilitates communication between designers and verifiers
 - Validates the verifier’s interpretation of the specifications
 - Enforces a common test framework, strategy, and style
 - Conserves resources due to less duplication of effort
- Design-unit test:
 - Owned by the designer/implementer
 - “Ad hoc”, quick and dirty, typically not communicated or maintained
 - Can be useful for preliminary “sanity” check



Ok, but when am I done?

25 © Cadence Design Systems, Inc. All rights reserved.



System-level test is about communication. The vast majority of the design errors that persist beyond a single phase of the design cycle do so, not because the information required to prevent them is not known to the team at large, but because the information is not known to the persons responsible for the piece of the design containing the error. System-level test is a formidable ally to the design team in their constant war against miscommunication or lack of communication.

Completion Criteria

There are two approaches:

- Test until time runs out (it will) – oops!
- Determine priorities and metrics before time runs out – follow them!
 - What tests [must / should / could] be implemented?
 - Manage time / goals / expectations.
 - Manage “feature creep”.
 - How do you know when you are done?
 - How good is “good enough”?

Count the ways...



An important part of the design verifier’s job is to provide meaningful project status information to the management team. The management team needs a reasonably firm test plan, against which they can analyze the project status trend, to do project-related planning. Although the test plan should retain some flexibility throughout the project, the design verifier should also make clear to the management team the time and resources trade-offs associated with any modifications.

Coverage Metrics

The most widely used completion metric is coverage.

(What percent of your verification goal is complete?)

- Code coverage – Coverage of the design code
 - Anything the simulation tools can automatically instrument and measure
 - Code blocks, expression terms, net toggles, FSM states and transitions
- Functional coverage – Coverage of the design *functionality*
 - User defines the coverage items
 - Data oriented – Which specified values, ranges of values, transitions between values occurred
 - Control oriented – Which specified sequences of control signals occurred

Verilog has no special constructs supporting coverage!



The most widely used completion metric is coverage. Coverage is in two forms:

- Code coverage is a measure of how completely the testbench exercises the design code. Code coverage includes any code items that simulation tools can automatically instrument and measure. This might include for example code blocks, expression terms, net toggles, and FSM states and transitions. Code coverage does not say anything about design functionality.
- Functional coverage is a measure of how completely the testbench exercises the design functionality. You the user have to specify this functionality, so the coverage metric is only as valid as the test specification. Functional coverage is further divided into two forms:
 - Data-oriented functional coverage is a measure of which specified values, ranges of values, and transitions between values occurred.
 - Control-oriented functional coverage is a measure of which specified sequences of control signals occurred.

Verilog has no special constructs supporting coverage. For code coverage that the tool automatically instruments this is not an issue. For functional coverage you have to turn to high level verification languages, such as SystemVerilog.

Example Code Coverage: Arbiter Model and Test

Code coverage provides an estimate of the upper limit of testbench quality.

<pre> module arb (output reg GNTA, GNTB, input wire REQA, REQB, RST, CLK); always @(posedge CLK) if (RST) {GNTA, GNTB} <= 0; else // UNIT A HAS PRIORITY case ({GNTA, GNTB}) 0: {GNTA, GNTB} <= {REQA, REQB && !REQA}; 1: {GNTA, GNTB} <= {REQA && !REQB, REQB}; 2: {GNTA, GNTB} <= {REQA, REQB && !REQA}; endcase endmodule </pre>	<pre> module arb_test; wire GNTA, GNTB; reg REQA, REQB, RST, CLK; arb u1 (GNTA, GNTB, REQA, REQB, RST, CLK); initial CLK=0; always #5 CLK=~CLK; initial begin @(negedge CLK) REQA=0; REQB=0; RST=0; @(negedge CLK) RST=1; // RESET @(negedge CLK) RST=0; @(negedge CLK) REQA=1; // REQ A @(negedge CLK) REQA=0; @(negedge CLK) REQB=1; // REQ B @(negedge CLK) REQB=0; @(negedge CLK) REQA=1; REQB=1; @(negedge CLK) REQA=0; @(negedge CLK) REQB=0; @(negedge CLK) \$finish; end endmodule </pre>
--	---



Code coverage provides an estimate of the upper limit of testbench quality. Block coverage, for example, indicates whether the covered code has been executed, and says nothing about how correct the code is. Code coverage is nonetheless very useful, for with fairly simple technology it can pinpoint an area of the design that the testbench has completely missed.

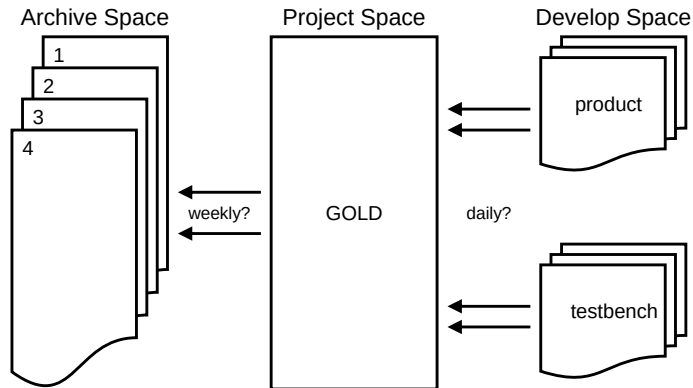
This arbiter simply responds with a grant to every request. The grant persists until the request drops. The arbiter gives priority to unit A if both units simultaneously make a request.

This short testbench resets the arbiter and then separately tests that it grants a unit A request and then a unit B request, and that upon simultaneous arrival of both requests, it grants the unit A request first.

Managing Design Space

Problem: Multiple parallel design efforts negatively impact each other.

Solution: Isolate design efforts by dividing and controlling design space.



Use source control to help manage your design space.

30 © Cadence Design Systems, Inc. All rights reserved.



Partition your design space into the following areas:

- 1st – The development space:
 - Copy elements to this private area for your individual design efforts;
 - Use the “golden” project space for all other design elements; and
 - Replace project space design elements only after thoroughly verifying them.
- 2nd – The project space:
 - Tag the in-progress elements to prevent duplication of design efforts;
 - Submit only thoroughly verified design elements; and
 - Continuously run regression tests.
- 3rd – The archive space:
 - Back up the project space here on a regular basis or upon completion of major features;
 - Continuously run the full test suite, incorporating random test sequences; and
 - Use this space for investigatory efforts such as timing analysis, fault analysis and so on...

Testing Design Regression

Characteristics

- Fully automated, usually cron-launched, shell scripts:
 - Control the what / where / when / how of system-level testing
 - Use good programming practices. Scripts are programs too!
 - Launch multiple simulations on multiple processors
 - Email a report with configurations, versions, errors, warnings

Concerns

- Implementers under schedule pressure check in minimally tested work.
- Implementers simultaneously check in tested, but incompatible, work.
- The regression cycle quickly exceeds the design modification cycle.
- Project management loses control of regression data. (What was *really* tested last week?)
- Regression testing *requires* testbenches to be self-checking!



You will want to continually in the background verify that addition of new features or modification of previous features does not cause test failure in the project space. This effort is termed “regression testing” or more confusingly just “regressions”. What this effort does is to detect the “regression” of the project to a less mature state.

Regression tests are usually fully automated cron-launched shell scripts that run multiple simulations on multiple processors, logging the tested configurations and versions, and emailing notification of any anomalies.

Regression testing of course requires that your testbenches be self-checking!

Self-Checking Test

There are two approaches:

- Visual compare (eyeballing) – after (or during!) simulation:
 - Ideal for lossy video processing (is it pretty enough?)
 - Very practical for simple functional tests of short duration
 - Generally not compatible with system-level test
- Automated compare (required for system-level test)
 - During simulation – Unattended run-time error handling
 - After simulation – Comparison of vector dump files may be sufficient

Examples

- Read-after-write or Cyclic Redundancy Character (CRC) checks
- Monitors of: Bus Protocols, Finite State Machine (FSM) states, timeouts
- Timing checks (as a form of assertion testing)
- Dump and compare of response vectors or memory contents

32

© Cadence Design Systems, Inc. All rights reserved.



Testbench self-checking is in two forms:

- The most common form is to check data and perhaps even protocols on the fly in order to provide immediate diagnostic information pointing to the area and time of the problem.
- A second less-common form is to automate a post-processing comparison of dumped test vectors to those of a known-good design. This form may be useful if you want a very detailed check of functionality that is very unlikely to have changed.

Your testbench can be self-checking in many ways:

- You can read back data that you write to storage devices.
- You can compare check characters calculated on the transmission side and the reception side.
- You can code monitors for bus protocols, FSM state paths, and timeouts.
- You can imaginatively set up timing checks to check the timing relationship between two edges of the same or different signals.

Example Self-Checking Test

```
`timescale 1 ns / 1 ns
module mux_test;
  localparam PERIOD=10;
  localparam EXIT_ON_ERROR=1;
  localparam STOP_ON_ERROR=0;
  localparam MUX_TEST_WAIT=PERIOD-1;
  wire y; reg a, b, s;
  // NEW MODEL BEING VERIFIED
  MUX2_1 mux ( y, s, b, a );
  // "GOLDEN" MODEL THAT WE KNOW WORKS
  wire results=y, expects=s?b:a;
  initial
  begin
    a=0; b=1; s=0; #(PERIOD);
    a=1; b=0; s=0; #(PERIOD);
    a=0; b=1; s=1; #(PERIOD);
    a=1; b=0; s=1; #(PERIOD);
    // A MISCOMPARE WON'T GET THIS FAR
    $display ("TEST PASSED");
    $finish;
  end
end
```

```
// FOR EACH MUX TEST EVENT
always @*
  // WAIT A PREDEFINED SETTLING TIME
  # (MUX_TEST_WAIT)
  // VERIFY THE RESULTS
  if (results != expects)
  begin
    $display($time, "ERROR MESSAGE");
    if (EXIT_ON_ERROR) $finish;
    if (STOP_ON_ERROR) $stop;
  end
endmodule
```

You first generate “expect” patterns using a model that you know works, such as:

- Your own knowledge of what the response should be at any given time.
- When modifying a model, the working model prior to modification.
- When generating a model of lower abstraction, the existing model of higher abstraction.

You then compare the model response to your previously generated “expect” patterns.

This example compares the results to “expect” patterns that a second model at a higher level of abstraction simultaneously generates. The known model of higher abstraction is a conditional expression.

A self-checking test may at first, before you fix it, falsely indicate miscomparisons:

- Perhaps due to different data types, especially if comparing data between two different simulators.
- Perhaps due to comparing data at different or unstable points in the clock cycle, especially when comparing a model with estimated timing to a simpler unit-delay model.
- Perhaps due to comparing data after different initialization sequences, as behavioral models often skip the prolonged initialization sequences the RTL and gate-level models must exercise.
- Perhaps due to comparing unknown or “don’t-care” data values.

Be careful to not confuse the logical equality (==) and case equality (===) operators. The logical equality operator produces an unknown result when comparing a high impedance or unknown state. The case equality operator always produces either a 1 or 0 result when comparing any two operands.

Example Self-Checking Test Alternative

```
`timescale 1 ns / 1 ns
module mux_test;
  localparam PERIOD=10;
  localparam EXIT_ON_ERROR=1;
  localparam STOP_ON_ERROR=0;
  wire y; reg a, b, s;

  // MODEL BEING VERIFIED
  MUX2_1 mux ( y, s, b, a );

  // VERIFY THE RESULTS
  task expect(input e);
    if (y != e)
      begin
        $display($time, "ERROR MESSAGE");
        if (EXIT_ON_ERROR) $finish;
        if (STOP_ON_ERROR) $stop;
      end
  endtask
```

```
// TEST THE MODEL
initial
  begin
    {s,b,a}=3'b010; #(PERIOD) expect(1'b0);
    {s,b,a}=3'b001; #(PERIOD) expect(1'b1);
    {s,b,a}=3'b110; #(PERIOD) expect(1'b1);
    {s,b,a}=3'b101; #(PERIOD) expect(1'b0);
    // A MISCOMPARE WON'T GET THIS FAR
    $display ("TEST PASSED");
    $finish;
  end
endmodule
```



This example utilizes the same model and test as previously, but instead of comparing the model results to those of a “golden” model, periodically calls a task to verify the model results.

Test Configuration

This section discusses:

- Test configuration in source code
- Test configuration with run-time scripts
- Test configuration in microcode



The following section discusses configuration of the test environment. It discusses test configuration in source code, with run-time scripts, in microcode, and with the Programming Language Interface (PLI).

Test Configuration Using Source Code Constructs

You can use source code constructs to statically configure your testbench:

- Select instances, tests, and tools to compile and invoke:
 - ``define`, ``ifdef`, `$test$plusargs`, `$value$plusargs`
- Specify configuration of system and testbench:
 - ``include` a file that you later link to one of a variety of design or test files
- Select level of abstraction on a per instance basis:
 - Use different cell names, for example, "asic_gate", "asic_rtl"
 - Use Verilog-2001 configurations to get cells from different libraries



As you will not be synthesizing the testbench, you can use the full range of Verilog constructs, choosing those that are most efficient or most easily coded for your application.

Some of the most common ways of presenting tests to the system are to put them in source code, run-time scripts, or pattern memory or microcode, perhaps applied via the PLI. Some of the methods you can use to configure the system and testbench in source code are:

- You can use the ``define` and ``ifdef` compiler directives to specify portions of the code to be compiled, thus selecting representations of instances, tests to be applied, and debug tools to be compiled into the simulation database.
- You can use the ``include` directive to include portions of code. The ``include` directive can itself be selected by the ``ifdef` directive, and the final resolution of the included file can be changed by modifying file-system links at the operating system level.
- You can use slightly different module names for different representations of any instantiated module, or retrieve the modules from different libraries.

Illustration of Test Configuration in Source Code (1)

```
// sys_config.h
`define CPU_BASIC
//`define CPU_QUAD
`define IO_BASIC
//`define IO_FAST
```

```
// system.v
`include "sys_config.h"
module system;
  `ifdef CPU_BASIC
    basic_cpu_pcb cpu(,,);
  `endif
  `ifdef CPU_QUAD
    quad_cpu_pcb cpu(,,);
  `endif
  `ifdef IO_BASIC
    basic_io_pcb io(,,);
  `endif
  `ifdef IO_FAST
    fast_io_pcb io(,,);
  `endif
  ...
endmodule
```

```
// test_config.h
`define THIS_TEST
`define THAT_TEST
//`define SOME_TEST
```

```
// test.v
`include "test_config.h"
module test;
  `ifdef THIS_TEST
    `include "this_test.v"
  `endif
  `ifdef THAT_TEST
    `include "that_test.v"
  `endif
  initial
  begin
    `ifdef THIS_TEST
      ;
    `endif
    `ifdef THAT_TEST
      that_test;
    `endif
  end
  ...
endmodule
```



This illustration shows how you can use text macros to configure the system under test. It uses text macros to select the system configuration and test configuration and launch the test tasks.

Test Configuration Using Microcode

You can use microcode to configure your test session:

- Write the testbench or DUT to utilize a memory model.
 - Can involve the PLI to dynamically allocate memory.
- Compile and initialize the simulation only once.
- Configure the test session statically or dynamically.
 - Statically – Precompile the test selection.
 - Dynamically – Interactively (or by script) select the tests to run.
 - Dynamically – Download test vectors during run time.
- Execute the test residing in the memory model.



Microcode execution is somewhat similar to vector playback. Both methods load and store test data, usually into pattern memory, which you may statically or dynamically allocate. The testbench can directly apply playback vectors to simulation objects, such as nets and registers. The testbench can generate and apply test vectors by executing microcode in a machine that is either in the testbench or part of the design under test. A microcoded test is usually significantly more compact than the same test represented with raw vectors.

Example of Test Configuration Using Microcode

```

module test ( output reg RST_, input CLK, HALT );

task run (input [7:0] test);
  reg [8*8:1] testfile;
  begin
    testfile = { "CPU", 8'h30+test, ".pat" }; // GENERATE TEST FILE NAME
    $readmemb ( testfile, top.cp.mem ); // LOAD TEST MICROCODE
  end
endtask

always @(posedge HALT) begin
  $display ( "Halted at address = %h", top.cp.pc ); // LAST TEST INSTRUCTION
  $restart ( "cpu_test.dat" ); // RESTART SIMULATION
end

initial begin
  @(negedge CLK) RST_ = 0;
  @(negedge CLK) RST_ = 1;
  @(negedge CLK) $save ( "cpu_test.dat" ); // SAVE SIMULATION STATE
  $display ( "CPU HALT" ); // SIMULATION RESTARTS HERE
  $write ( "To run n'th test enter:" );
  $write ( "\ttb.run(n);\n" );
  $stop;
end

endmodule

```

For example:
"CPU0.pat"

39 © Cadence Design Systems, Inc. All rights reserved.

cadence

In this example, the testbench saves the entire simulation database immediately upon completing the initialization. The initialization sequence for a typical design could require significant simulation time that may not be necessary to repeat for each subtest.

This example generates a unique filename from the “run” task argument, and reads the vectors in that file into memory to exercise the CPU model.

The \$restart system task reloads the simulation database. The state of the simulation upon restarting is exactly the state saved by the \$save system task.

The IEEE Standard 1364-2001 Verilog describes but does not require the \$save and \$restart system tasks. Not all simulators support these tasks, and some offer this feature with some other proprietary interactive command.

To run these examples “as is”, the simulator must retain an interpretive engine. Purely compiled-code simulators cannot interpret Verilog code entered after the simulation starts.

Program Code Which Runs the Test

```

/***** CPU0.pat *****/
* Test program for the VeriRISC CPU system
*
* This diagnostic program tests the basic instruction set of the VeriRISC system. If the system executes each
* instruction correctly, then it should halt when the HLT instruction at address 17(hex) is executed.
* If the system halts at any other location, then an instruction did not execute properly. Refer to the
* comments in this file to see which instruction failed.
*****/
//opcode_operand // addr assembly code
-----
000 111_11110 // 00 BEGIN: JMP TST_JMP
000_00000 // 01 HLT //JMP did not work at all
000_00000 // 02 HLT //JMP did not load PC, it skipped
101_11010 // 03 JMP_OK: LDA DATA_1
001_00000 // 04 SKZ
000_00000 // 05 HLT //SKZ or LDA did not work
101_11011 // 06 LDA DATA_2
001_00000 // 07 SKZ
111_01010 // 08 JMP SKZ_OK
000_00000 // 09 HLT //SKZ or LDA did not work
110_11100 // 0A SKZ_OK: STO TEMP //store non-zero value in TEMP
101_11010 // 0B LDA DATA_1
110_11100 // 0C STO TEMP //store zero value in TEMP
101_11100 // 0D LDA TEMP
001_00000 // 0E SKZ //check to see if STO worked
000_00000 // 0F HLT //STO did not work
100_11011 // 10 XOR DATA_2
001_00000 // 11 SKZ //check to see if XOR worked
111_10100 // 12 JMP XOR_OK
000_00000 // 13 HLT //XOR did not work at all
100_11011 // 14 XOR_OK: XOR DATA_2
001_00000 // 15 SKZ
000_00000 // 16 HLT //XOR did not switch all bits
000_00000 // 17 END: HLT //CONGRATULATIONS - TEST1 PASSED!
111_00000 // 18 JMP BEGIN //run test again

@1A 00000000 // 1A DATA_1: //constant 00(hex)
11111111 // 1B DATA_2: //constant FF(hex)
10101010 // 1C TEMP: //variable - starts with AA(hex)

@1E 111_00011 // 1E TST_JMP: JMP JMP_OK
000_00000 // 1F HLT //JMP is broken

```

This page does not contain notes.

Module Summary

You should now be able to optimize testbench development and create reusable testbenches.

This module discussed:

- Test philosophy:
 - Verification challenges, plans, and goal
 - System-level test
 - Coverage metrics
 - Managing design space
 - Testing design regression
 - Self-checking test
- Test configuration:
 - Using source code constructs
 - Using microcode



You should now be able to optimize testbench development and create reusable testbenches.

This module presented design verification concepts, reinforced with small working examples and illustrations. The first part of this module explored the philosophy of system-level test, including the test plan, completion criteria, and design data management. The second part described self-checking test methods and various ways to configure the test environment.